

PYTHON



Julia Muñoz Ruiz



JULIA MUÑOZ RUIZ

RESPONSABLE DEL DESARROLLO DE LA APLICACIÓN

Fecha: 12, MARZO, 2026



ÍNDICE DE CONTENIDOS

1. DESCRIPCIÓN GENERAL DEL PROYECTO	1
1. Marco teórico	1
2. Funcionamiento del experimento	1
3. Estructura de la aplicación	1
2. OBJETIVOS Y ALCANCE DEL PROYECTO	2
1. Objetivos principales	2
2. Alcance del proyecto	2
3. Limitaciones conocidas	3
3. STACK TECNOLÓGICO Y ALTERNATIVAS EVALUADAS	5
1. Flask vs Django vs FastAPI	5
2. Bases de datos SQL y NoSQL	6
1. SQLite vs PostgreSQL vs MySQL	6
3. Tecnologías complementarias	7
4. MODELO DE DATOS	8
1. Tabla Participantes	8
2. Tabla Lenguas	9
3. Tabla Materiales	11
4. Tabla Respuestas	12
5. Diagrama de relaciones	13
1. Participantes 1:N	13
2. Materiales 1:N	13
3. Lenguas	14
6. Vistas (<i>views</i>) en DB Browser	16
1. Porcentaje de acierto significado-audio	16



2. Porcentaje de acierto por ítem	17
3. Porcentaje de acierto por lengua	18
5. REQUISITOS DE LA APLICACIÓN	19
1. Requisitos funcionales	19
2. Requisitos no funcionales	22
3. Requisitos del entorno	23
6. MANUAL DE INSTALACIÓN	23
1. Requisitos previos	23
2. Instalación paso a paso	24
3. Configuración de la Base de Datos (BBDD)	28
4. Ejecución del experimento	30
5. Solución de problemas comunes	32
1. Error: No Interpreter Configured	32
2. Error: Module Not Found (Flask, SQLAlchemy)	32
3. Error: Puerto 500 ya en uso	32
4. Error: No se reproduce el audio	33
5. Error: Base de datos bloqueada	33
6. PyCharm no encuentra la carpeta del proyecto	33
7. Conclusiones	33
8. Evolutivo del proyecto	34



1. DESCRIPCIÓN GENERAL DEL PROYECTO

El objetivo de este proyecto ha sido el desarrollo de una página web cuyo uso está enfocado en la investigación psicolingüística. El contexto de investigación es la iconicidad en la deixis espacial, es decir, la relación no arbitraria entre las características fonéticas de las pseudopalabras creadas y su significado deíctico espacial.

1.1 Marco teórico

En rasgos generales, la investigación está basada en la teoría del simbolismo sonoro, que sugiere que ciertos fonemas están asociados intrínsecamente con significados espaciales específicos. De acuerdo a estudios tipológicos previos (Woodworth, 1991; Traunmüller, 1994; Johansson & Zlatev, 2013) y basado en la teoría del Código de Frecuencia (Ohala, 1994), existe una relación sistemática entre:

- vocales de alta frecuencia en su segundo formante (F2) y significados proximales: la vocal [i] – cerca del hablante;
- vocales de baja frecuencia F2 ([u], [a]) y significados distales (lejos del hablante);
- consonantes sordas ([t], [p], [k], [s]) y significados proximales, y
- consonantes sonoras ([b], [n], [m], [l]) y significados distales.

1.2 Funcionamiento del experimento

El experimento en sí presenta pseudopalabras (como “sipi”, “nuba”, “satu”) creadas de acuerdo a estas normas icónico-fonéticas a los participantes, y estos deben asociarlas con significados deícticos espaciales (“aquí”, “allí”, “este”, “aquel”). Los estímulos auditivos se generaron de forma natural mediante la grabación de voz humana para mantener la naturalidad de la pronunciación. Para consultar la tabla de materiales, ir al punto 4.3.

1.3 Estructura de la aplicación

La aplicación web consta de la siguientes páginas:

- formulario de registro: captura de datos sociodemográficos y perfil lingüístico del participante,



- instrucciones: dos páginas explicativas sobre el procedimiento del experimento;
- experimento: presentación secuencial de pseudopalabras con opciones de respuesta, y
- finalización: pantalla de agradecimiento que indica que se ha completado la actividad.

Todos los datos (tanto la información de los participantes como sus respuestas) se almacenan en una base de datos SQLite local para su posterior análisis estadístico.

2. OBJETIVOS Y ALCANCE DEL PROYECTO

2.1 Objetivos principales

Los objetivos principales del proyecto son:

- 1) desarrollar una herramienta web para la recolección de datos experimentales sobre simbolismo sonoro en deixis espacial,
- 2) implementar un sistema de presentación de estímulos auditivos controlados con pseudopalabras diseñadas de acuerdo a criterios fonéticos específicos,
- 3) crear una base de datos robusta para almacenar perfiles lingüísticos de participantes y sus respuestas experimentales, y
- 4) permitir la randomización de estímulos para controlar efectos de orden.

2.2 Alcance del proyecto

Al estar concebido para un uso en entorno académico o de laboratorio, su entorno de ejecución es *localhost* (servidor local), con almacenamiento en una base de datos SQLite en un único archivo. El acceso al experimento es controlado ya que se usa un USB o la red local del laboratorio, y no existen usuarios *per se*, ya que los participantes individuales están bajo supervisión del investigador. Por este método de uso, no es necesario la creación de un *log in* y el control de acceso está cubierto mediante el registro de los datos de los participantes.



2.3 Limitaciones conocidas

De la misma forma, sus limitaciones son la ausencia de un sistema de autenticación o gestión de usuarios a nivel externo, no está optimizado para desplegarlo en producción con múltiples usuarios concurrentes, los estímulos auditivos deben estar pregenerados ya que no hay síntesis de audio en tiempo real, y la interfaz está únicamente en español.

3. STACK TECNOLÓGICO Y ALTERNATIVAS EVALUADAS

El *stack* tecnológico principal utilizado son Python, Flask y SQLite.

3.1 Flask vs Django vs FastAPI

Al comparar el *framework* Flask con Django y FastAPI, encontramos que los beneficios de Flask como *stack* tecnológico son superiores al de los otros dos mencionados. Esto se debe a que Flask es simple y minimalista (las bibliotecas, extensiones y demás necesidades que puedan surgir quedan a cargo del desarrollador), lo que lo hace ideal para proyectos pequeños como este, permite desarrollo rápido sin configuración compleja ya que tiene una curva de aprendizaje suave, sólo incluye lo necesario (sin componentes innecesarios) por lo que es flexible, tiene una gran documentación y una comunidad *online* activa en caso de dudas, y se integra de forma fácil con SQLAlchemy para ORM.

Por el contrario, Django incluye un sistema de autenticación, admin y ORM¹ complejo que no son necesarios en este proyecto, su estructura es más rígida, lo que le da mayor complejidad, y su configuración inicial es mucho más extensa para un proyecto simple como este, por lo que el tiempo de desarrollo es demasiado extenso para algo que puede conseguirse de forma más sencilla. De la misma forma, FastAPI tiene una complejidad innecesaria para este caso de uso por su programación asíncrona, tiene menor madurez en renderizado de plantillas en comparación a Flask o Jinja2, y su enfoque en APIs REST lo descarta automáticamente por la necesidad del proyecto del renderizado de HTML, no solo API.

¹ *Object-Related Mapping* o Mapeo relacional de objetos. Es decir, dentro de la programación orientada a objetos (POO) es un proceso que consiste en emparejar valores en bases de datos relacionales. Se encarga de hacer consultas e interactuar con la base de datos usando el lenguaje de *backend* y de hacer el mapeo de las estructuras de las bases de datos a objetos.



3.2 Bases de datos SQL vs NoSQL

Para el tratamiento de base de datos, se evaluó el uso de NoSQL (como MongoDB) y SQL (MySQL, PostgreSQL, SQLite). Por un lado, las bases de datos NoSQL almacenan datos en formatos flexibles (documentos JSON, pares clave-valor, etc.) sin esquema rígido, y están diseñadas para tener escalabilidad horizontal y con datos semi-estructurados o heterogéneos. Por otro lado, las bases de datos SQL organizan los datos en tablas relacionales con esquemas predefinidos y garantizan integridad referencial mediante *Foreign Keys*, además de soportar transacciones ACID (Atomicidad, Consistencia, Aislamiento y Durabilidad).

En este caso, las NoSQL no son aplicables en este proyecto porque los datos experimentales son altamente estructurados y relacionales (participantes, respuestas, materiales), es necesario mantener la integridad referencial estricta, ya que las respuestas no pueden existir sin los participantes, el análisis estadístico necesita consultas complejas con *JOINS* y agregaciones, y no se requiere escalabilidad masiva, dado que los participantes se miden en decenas en un entorno local. Por el contrario, las SQL son más apropiadas ya que las tablas reflejan perfectamente las entidades del experimento y su esquema estructurado, se garantiza la integridad mediante *constraints* y *Foreign Keys* que previenen datos inconsistentes, las consultas son analíticas al permitir agrupar, filtrar y agregar datos eficientemente, tiene portabilidad y un ecosistema maduro basado en herramientas gráficas (DB Browser), librerías de análisis (pandas, R) y software estadístico (SPSS) que se integran directamente con SQL si fuera necesario.

3.2.1 SQLite vs PostgreSQL vs MySQL

Dentro de este tipo de bases de datos, consideramos PostgreSQL, MySQL y SQLite. Así, mientras que PostgreSQL y MySQL requieren instalación y configuración del servidor, tienen mayor complejidad de gestión y *back-up*, dificultad para su portabilidad a través del USB y una sobrecapacidad para el volumen de datos con los que vamos a trabajar, SQLite no requiere un servidor separado ni configuración, tiene mayor portabilidad al almacenarse en un único archivo .db, lo que también simplifica el *back-up*, no requiere administración (ideal para un entorno de laboratorio controlado), y tiene un rendimiento suficiente para el volumen de datos del experimento (participantes en decenas).



3.3 Tecnologías complementarias

Las tecnologías complementarias en el *backend* son SQLAlchemy, Jinja2, Bootstrap 5 y Select2.

El primero, SQLAlchemy, es un ORM (definición en el pie de página 1) que permite interactuar con la base de datos mediante objetos Python sin escribir SQL directamente, además, facilita la creación de modelos de datos y consultas.

El segundo, Jinja2) es un motor de plantillas integrado en Flask que permite crear HTML dinámico con lógica de presentación (como bucles o condicionales) dentro de las propias plantillas HTML.

El tercero, Bootstrap 5, es un *framework* de CSS que sirve para un diseño *responsive*² y componentes de interfaz (formularios, botones, *grid*), en nuestro proyecto seleccionamos el tema “Simplex” de Bootswatch para dar un aspecto visual limpio y profesional.

El cuarto, Select2, es un *plugin* de jQuery para mejorar los selectores HTML (la selección de nivel educativo, por ejemplo), con especial atención al selector multilingüe donde los participantes indican las lenguas que conocen.

Por último, para el tratamiento de la base de datos utilizamos DB Browser para SQLite, que es una herramienta gráfica gratuita y multiplataforma para gestionar bases de datos SQLite. Este permite visualizar y editar datos de tablas de forma directa, ejecutar consultas SQL personalizada, crear y modificar la estructura de la tabla, exportar datos a CSV, JSON o SQL para análisis externo, crear vistas (*views*) para consultas frecuentes, y visualizar el esquema completo de la base de datos. En este proyecto, se utiliza principalmente para poblar el catálogo inicial de lenguas, verificar datos de participantes y respuestas tras cada sesión experimental, y para crear vistas SQL para consultas analíticas recurrentes.

² Un formato de programación que permite ajustar un sitio web automáticamente al tamaño y disposición de los dispositivos de sus usuarios. Así, los sitios web cambian para mejorar la experiencia de los usuarios de acuerdo al medio usado: *smartphones*, tabletas u ordenadores (portátiles o de escritorio).



4. MODELO DE DATOS

La base de datos consta de cuatro (4) tablas relacionales que almacenan información de participantes, catálogos de lenguas y materiales experimentales, y las respuestas registradas en el experimento.

4.1 Tabla Participantes

Así, la tabla Participantes almacena el perfil socio-demográfico y lingüístico de cada participante como se muestra en la tabla:

Campo	Tipo	Restricciones	Descripción
id	Integer	PRIMARY KEY	Identificador único autoincremental
nombre	String(200)	NOT NULL	Nombre del participante
apellidos	String(200)	NOT NULL	Apellidos del participante
edad	Integer	NOT NULL	Edad del participante
nivel_estudios	String(200)	NOT NULL	Nivel educativo (ESO, Bachillerato, Grado, etc.)
lengua_materna	String	NOT NULL	Lengua materna del participante
otras_lenguas	String	NULL	Lista separada por “;” de otras lenguas conocidas



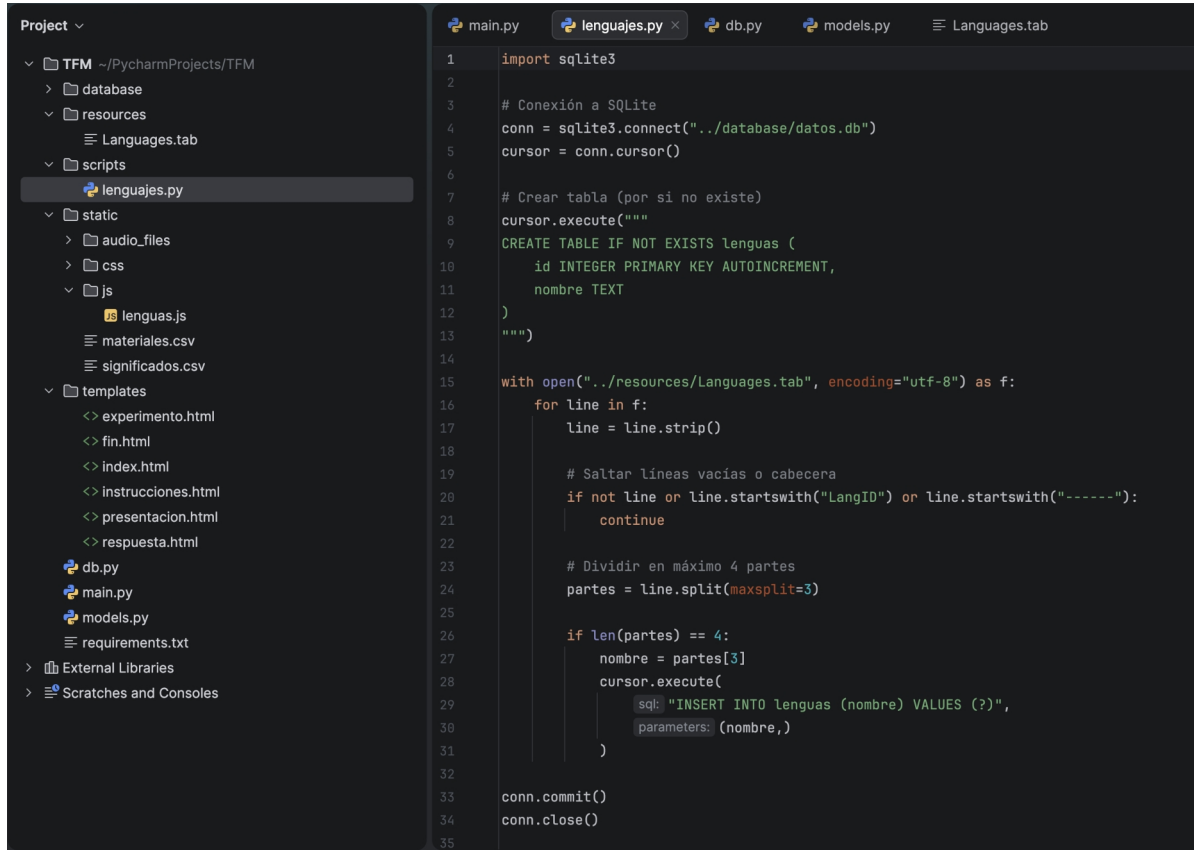
4.2 Tabla Lenguas

La tabla Lenguas es un catálogo de lenguas disponibles para su selección en el formulario, ya sea como lengua materna o como otras lenguas habladas por el participante.

Campo	Tipo	Restricciones	Descripción
id	Integer	PRIMARY KEY	Identificador único autoincremental
nombre	String(200)	NOT NULL	Nombre de la lengua en inglés (ej: Euskera, Spanish, English)

Es importante saber que esta tabla de lenguas ha sido importada directamente de la página ETHNOLOGUE³, y el para crear la tabla SQL código puede verse en “/scripts/lenguajes.py”.

³Web de ETHNOLOGUE: <https://www.ethnologue.com/codes/>



```
1 import sqlite3
2
3 # Conexión a SQLite
4 conn = sqlite3.connect("../database/datos.db")
5 cursor = conn.cursor()
6
7 # Crear tabla (por si no existe)
8 cursor.execute("""
9 CREATE TABLE IF NOT EXISTS lenguas (
10     id INTEGER PRIMARY KEY AUTOINCREMENT,
11     nombre TEXT
12 )
13 """)
14
15 with open("../resources/Languages.tab", encoding="utf-8") as f:
16     for line in f:
17         line = line.strip()
18
19         # Saltar líneas vacías o cabecera
20         if not line or line.startswith("LangID") or line.startswith("-----"):
21             continue
22
23         # Dividir en máximo 4 partes
24         partes = line.split(maxsplit=3)
25
26         if len(partes) == 4:
27             nombre = partes[3]
28             cursor.execute(
29                 sql: "INSERT INTO lenguas (nombre) VALUES (?)",
30                 parameters: (nombre,)
31             )
32
33 conn.commit()
34 conn.close()
35
```



4.3 Tabla Materiales

La tabla Materiales contiene los estímulos experimentales: las pseudopalabras, sus opciones de respuesta y el material de audio al que tiene que llamar el programa.

Campo	Tipo	Restricciones	Descripción
id	Integer	PRIMARY KEY	Identificador único autoincremental
id_item	String(200)	NOT NULL	Código del ítem (ej: ITEM01, ITEM015)
audio_material	String(200)	NOT NULL	Nombre del archivo de audio (ej: si.mp3, nuba.mp3)
sign_esp	String	NOT NULL	Significado esperado (cerca, medio, lejos)
opt_a	String	NOT NULL	Texto de la opción A
sign_a	String	NOT NULL	Significado de la opción A
opt_b	String	NOT NULL	Texto de la opción B
sign_b	String	NOT NULL	Significado de la opción B
opt_c	String	NOT NULL	Texto de la opción C
sign_c	String	NOT NULL	Significado de la opción C
opt_d	String	NOT NULL	Texto de la opción D
sign_d	String	NOT NULL	Significado de la opción D



4.4 Tabla Respuestas

La tabla Respuestas registra las respuestas de cada participante a cada ítem individual del experimento.

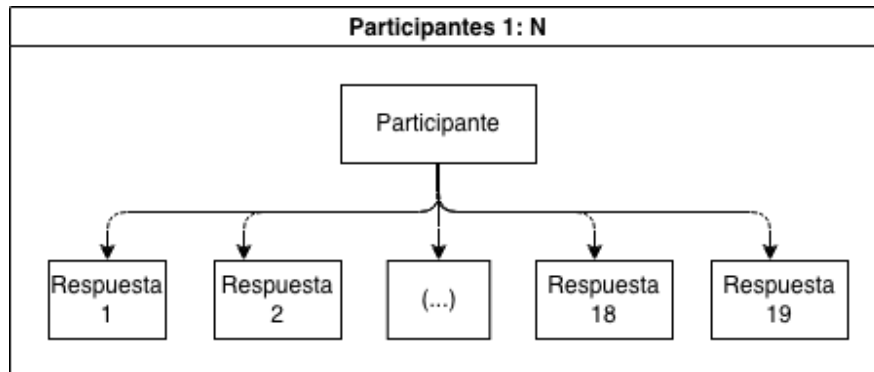
Campo	Tipo	Restricciones	Descripción
id	Integer	PRIMARY KEY	Identificador único autoincremental
id_item	String(200)	NOT NULL	Código del ítem respondido
id_participante	Integer	NOT NULL	ID del participante (FK a Participantes)
sign_esp	String(200)	NOT NULL	Significado esperado del ítem
opt_resp	String	NOT NULL	Opción elegida por el participante
sign_resp	String	NOT NULL	Significado de la opción elegida
ok	Integer	NOT NULL	Indicador de acierto (1=correcto, 0=incorrecto)

4.5 Diagrama de relaciones

Las relaciones entre tablas son las siguientes:

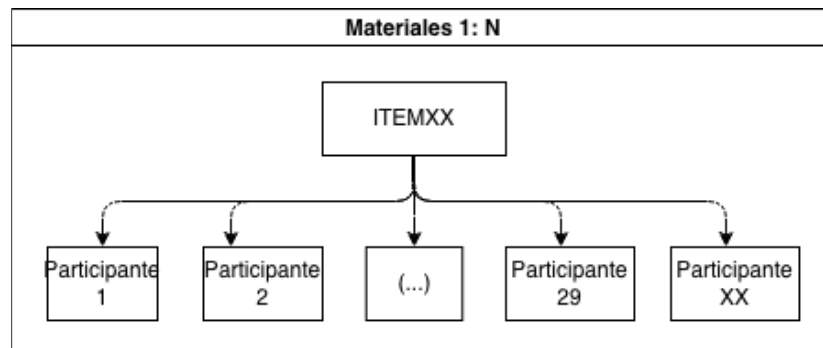
4.5.1 Participantes 1:N

Un participante genera múltiples respuestas.



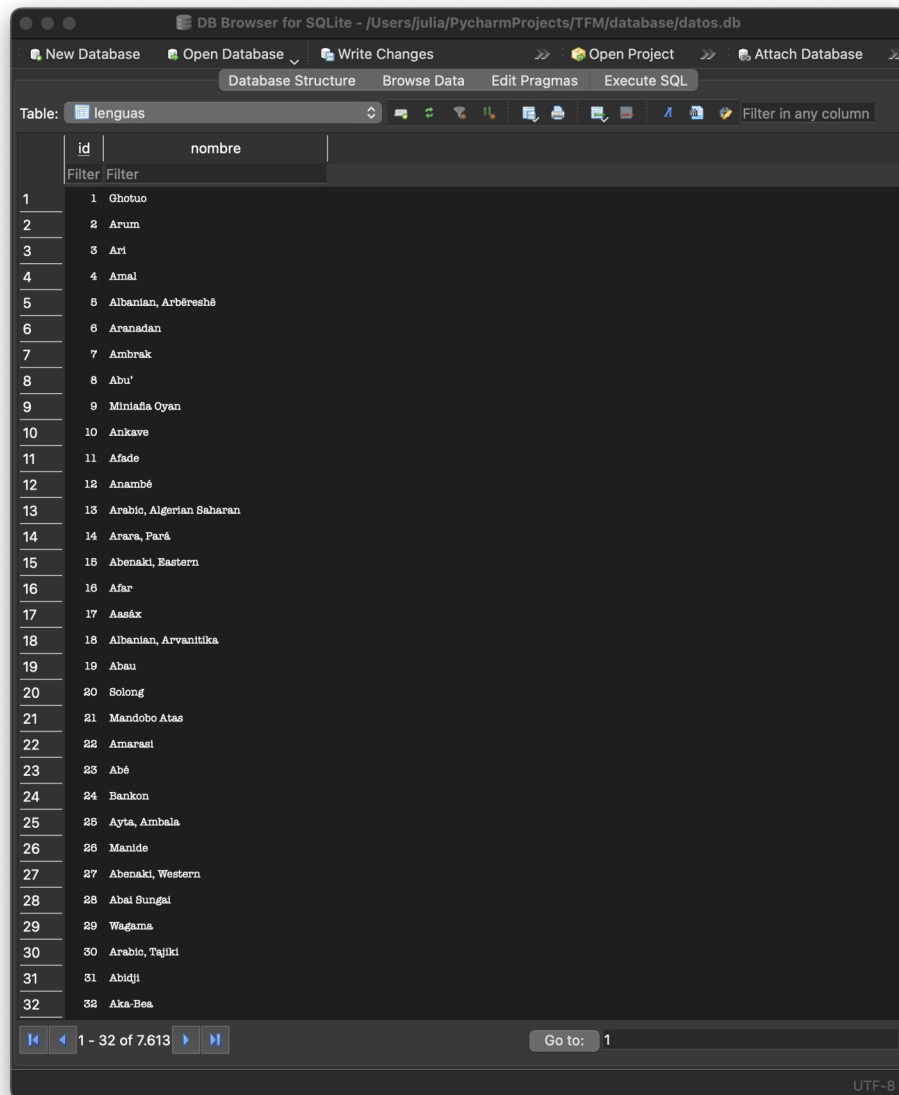
4.5.2 Materiales 1:N

Cada ítem puede recibir múltiples respuestas de diferentes participantes.



4.5.3 Lenguas

Se trata de una tabla de referencia sin relaciones explícitas usada para poblar sectores. El catálogo de lenguas se nutre inicialmente desde el archivo “/resources/languages.tab”, que contiene una lista predefinida de idiomas (en inglés).



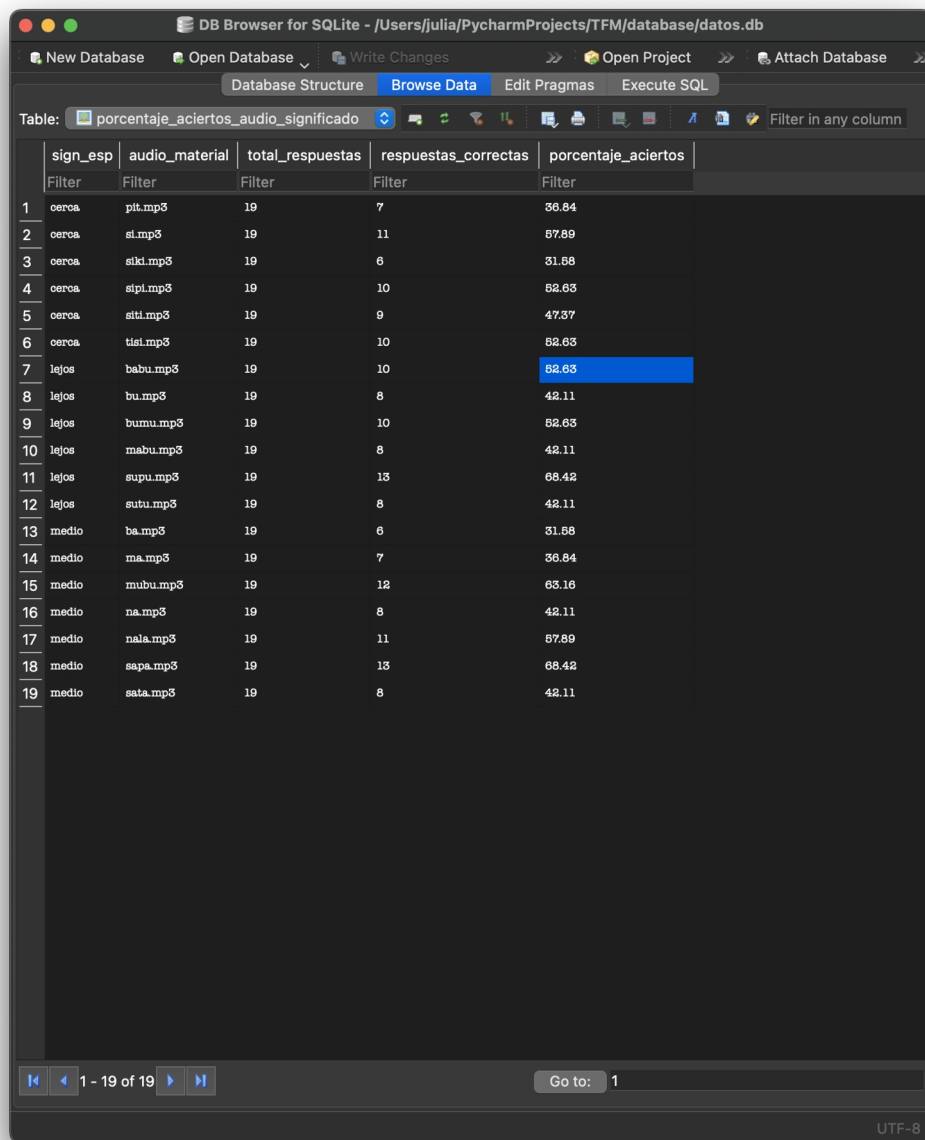
	id	nombre
1	1	Ghotuo
2	2	Arum
3	3	Ari
4	4	Amal
5	5	Albanian, Arbëreshë
6	6	Aranadan
7	7	Ambrak
8	8	Abu'
9	9	Miniafa Oyan
10	10	Ankave
11	11	Afade
12	12	Anambé
13	13	Arabic, Algerian Saharan
14	14	Arara, Pará
15	15	Abenaki, Eastern
16	16	Afar
17	17	Aasáx
18	18	Albanian, Arvanitika
19	19	Abau
20	20	Solong
21	21	Mandobo Atas
22	22	Amarasi
23	23	Abé
24	24	Bankon
25	25	Ayta, Ambala
26	26	Manide
27	27	Abenaki, Western
28	28	Abai Sungai
29	29	Wagama
30	30	Arabic, Tajiki
31	31	Abidji
32	32	Aka-Bee

4.6 Vistas (*views*) en DB Browser

Con estas tablas, creamos en DB Browser tres vistas diferentes: el porcentaje de acierto en el significado por audio, el porcentaje de aciertos en el significado por ítem, el porcentaje de acierto de acuerdo a la lengua materna.

4.6.1 Porcentaje de acierto significado-audio

Esta vista se centra en la relación entre el audio y su significado esperado, y nos muestra el porcentaje de acierto que existe entre ambos. Así, nos centramos en la relación de los fonemas con el significado asociado.

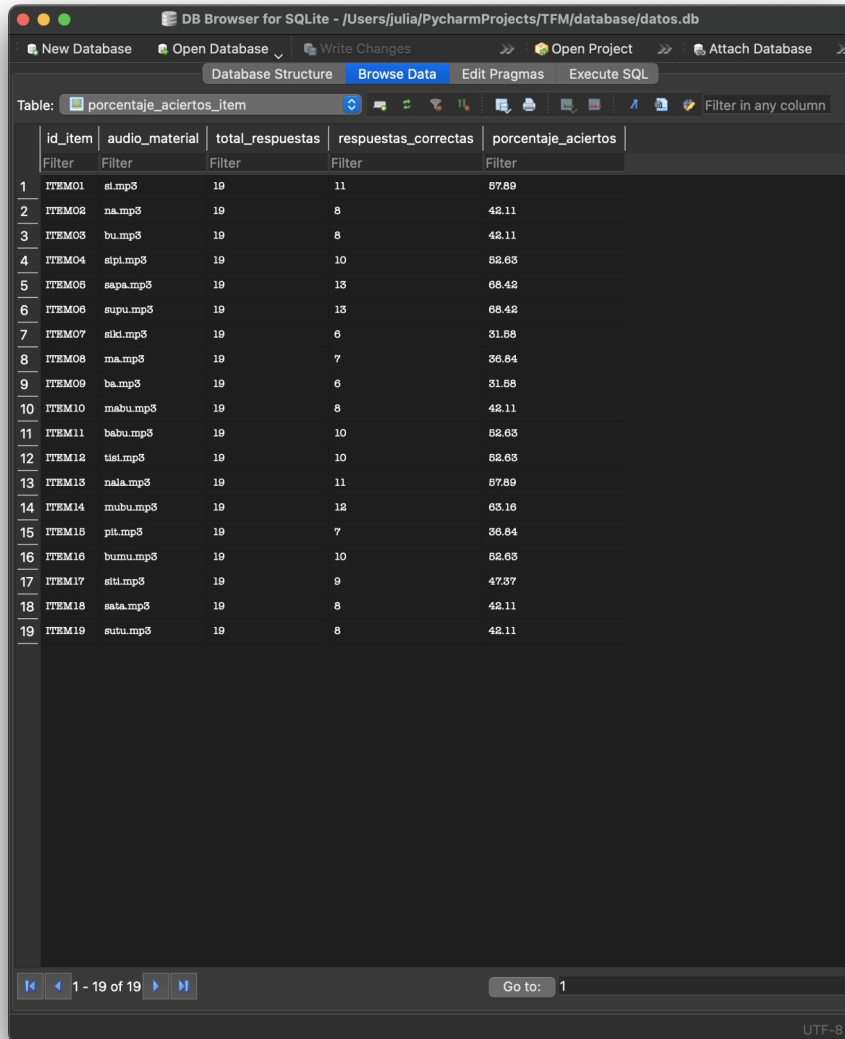


The screenshot shows the DB Browser for SQLite interface. The table 'porcentaje_aciertos_audio_significado' is displayed with the following data:

	sign_esp	audio_material	total_respuestas	respuestas_correctas	porcentaje_aciertos
	Filter	Filter	Filter	Filter	Filter
1	ceroa	pit.mp3	19	7	36.84
2	ceroa	si.mp3	19	11	57.89
3	ceroa	siki.mp3	19	6	31.58
4	ceroa	sipi.mp3	19	10	52.63
5	ceroa	siti.mp3	19	9	47.37
6	ceroa	tisi.mp3	19	10	52.63
7	lejos	babu.mp3	19	10	52.63
8	lejos	bu.mp3	19	8	42.11
9	lejos	bumu.mp3	19	10	52.63
10	lejos	mabu.mp3	19	8	42.11
11	lejos	supu.mp3	19	13	68.42
12	lejos	sutu.mp3	19	8	42.11
13	medio	ba.mp3	19	6	31.58
14	medio	ma.mp3	19	7	36.84
15	medio	mubu.mp3	19	12	63.16
16	medio	na.mp3	19	8	42.11
17	medio	naia.mp3	19	11	57.89
18	medio	sapa.mp3	19	13	68.42
19	medio	sata.mp3	19	8	42.11

4.6.2 Porcentaje de acierto por ítem

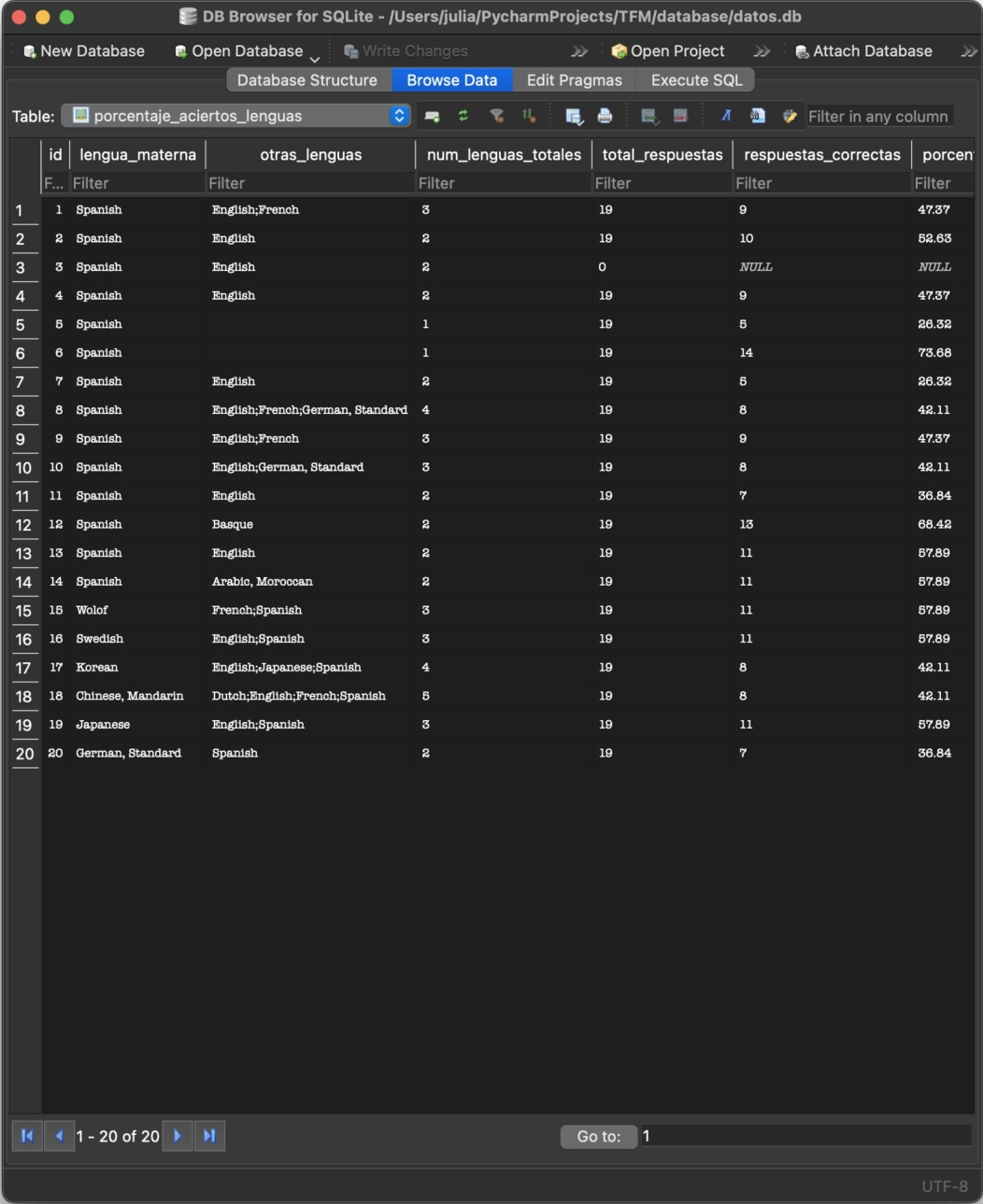
Esta vista se centra en el ítem y el audio, ignorando el significado, para poder contabilizar qué ítems tienen mayor porcentaje de acierto o de error. Así, nos centramos en las características fonéticas propias y cuáles han tenido mayor acierto.



	id_item	audio_material	total_respuestas	respuestas_correctas	porcentaje_aciertos
	Filter	Filter	Filter	Filter	Filter
1	ITEM01	sl.mp3	19	11	57.89
2	ITEM02	na.mp3	19	8	42.11
3	ITEM03	bu.mp3	19	8	42.11
4	ITEM04	stpl.mp3	19	10	52.63
5	ITEM06	sapa.mp3	19	13	68.42
6	ITEM06	supu.mp3	19	13	68.42
7	ITEM07	stkl.mp3	19	6	31.58
8	ITEM08	ma.mp3	19	7	36.84
9	ITEM09	ba.mp3	19	6	31.58
10	ITEM10	mabu.mp3	19	8	42.11
11	ITEM11	babu.mp3	19	10	52.63
12	ITEM12	tisl.mp3	19	10	52.63
13	ITEM13	naia.mp3	19	11	57.89
14	ITEM14	mubu.mp3	19	12	63.16
15	ITEM15	pil.mp3	19	7	36.84
16	ITEM16	bumu.mp3	19	10	52.63
17	ITEM17	stti.mp3	19	9	47.37
18	ITEM18	sata.mp3	19	8	42.11
19	ITEM19	sutu.mp3	19	8	42.11

4.6.1 Porcentaje de acierto por lengua

Esta vista se centra en las lenguas maternas, además de mostrar otras posibles lenguas que hable cada participante. Así, podemos observar si ciertas lenguas tienen mayor o menor sensibilidad a la iconicidad déctica.



DB Browser for SQLite - /Users/julia/PycharmProjects/TFM/database/datos.db

Database Structure Browse Data Edit Pragma Execute SQL

Table: porcentaje_aciertos_lenguas

	id	lengua_materna	otras_lenguas	num_lenguas_totales	total_respuestas	respuestas_correctas	porcen
	F...	Filter	Filter	Filter	Filter	Filter	Filter
1	1	Spanish	English;French	3	19	9	47.37
2	2	Spanish	English	2	19	10	52.63
3	3	Spanish	English	2	0	NULL	NULL
4	4	Spanish	English	2	19	9	47.37
5	5	Spanish		1	19	5	26.32
6	6	Spanish		1	19	14	73.68
7	7	Spanish	English	2	19	5	26.32
8	8	Spanish	English;French;German, Standard	4	19	8	42.11
9	9	Spanish	English;French	3	19	9	47.37
10	10	Spanish	English;German, Standard	3	19	8	42.11
11	11	Spanish	English	2	19	7	36.84
12	12	Spanish	Basque	2	19	13	68.42
13	13	Spanish	English	2	19	11	57.89
14	14	Spanish	Arabic, Moroccan	2	19	11	57.89
15	15	Wolof	French;Spanish	3	19	11	57.89
16	16	Swedish	English;Spanish	3	19	11	57.89
17	17	Korean	English;Japanese;Spanish	4	19	8	42.11
18	18	Chinese, Mandarin	Dutch;English;French;Spanish	5	19	8	42.11
19	19	Japanese	English;Spanish	3	19	11	57.89
20	20	German, Standard	Spanish	2	19	7	36.84

1 - 20 of 20 Go to: 1

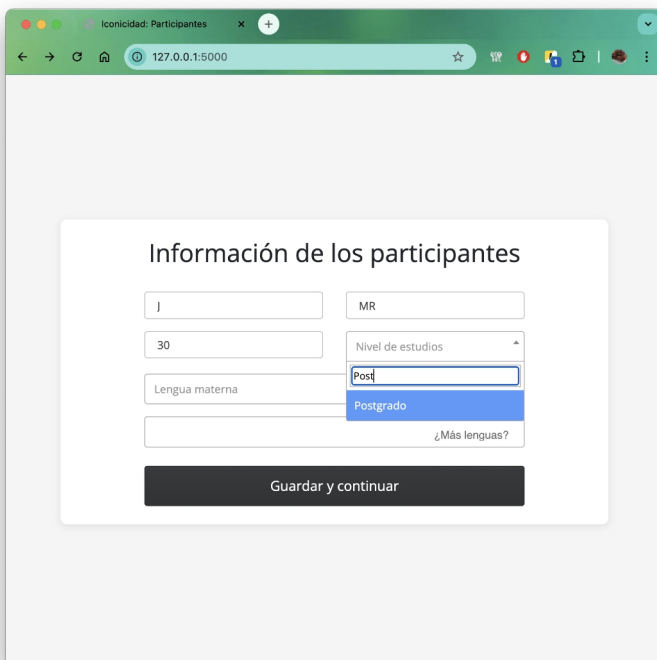
UTF-8

5. REQUISITOS DE LA APLICACIÓN

5.1 Requisitos funcionales

Los requisitos funcionales de la página web incluyen el registro de participantes, la presentación de instrucciones, la ejecución del experimento, el almacenamiento de datos y el manejo de sesiones.

El registro de participantes incluye capturar el nombre completo, la edad, el nivel de estudios (selector desplegable), la lengua materna (selector con búsqueda) y otras lenguas conocidas por el participante (selector múltiple con búsqueda).



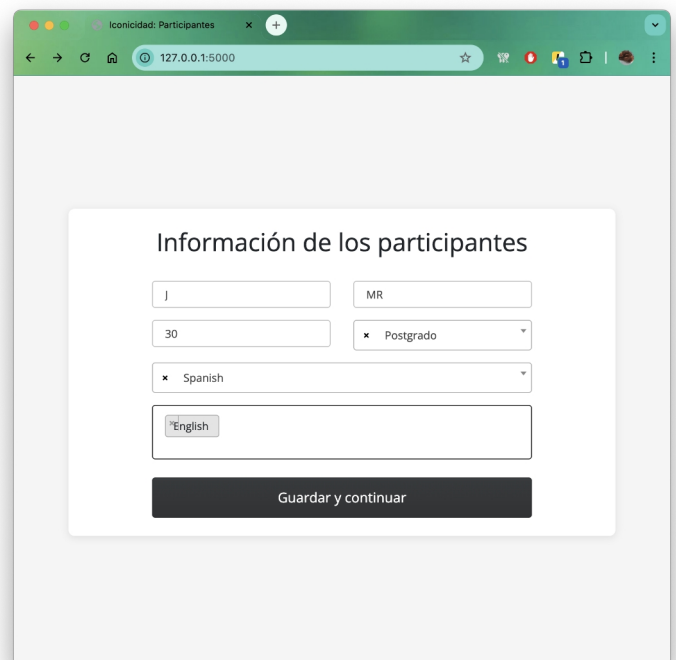
Iconicidad: Participantes x +

127.0.0.1:5000

Información de los participantes

J	MR
30	Nivel de estudios
Lengua materna	Post
	Postgrado
	¿Más lenguas?

Guardar y continuar



Iconicidad: Participantes x +

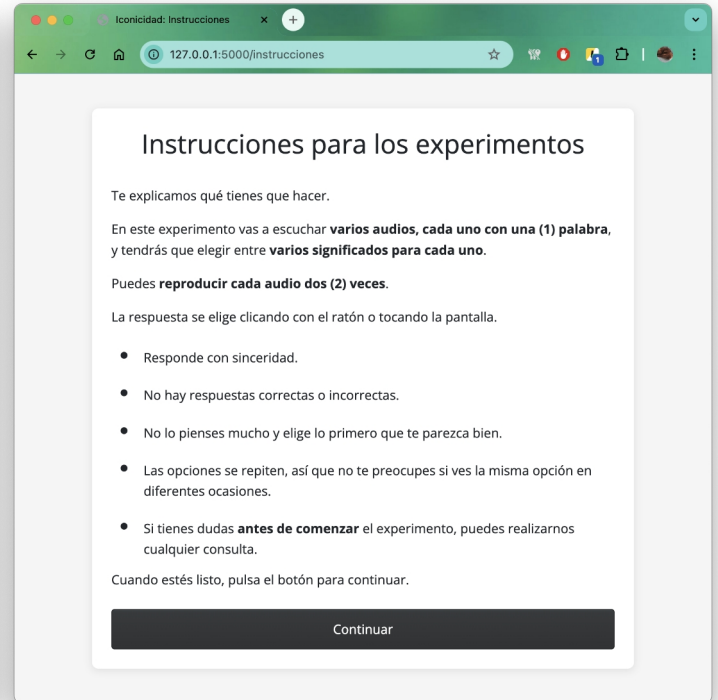
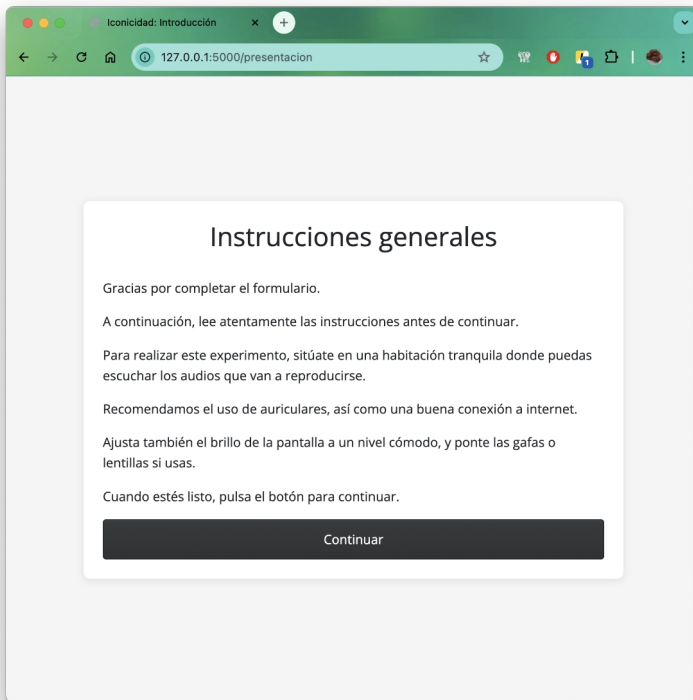
127.0.0.1:5000

Información de los participantes

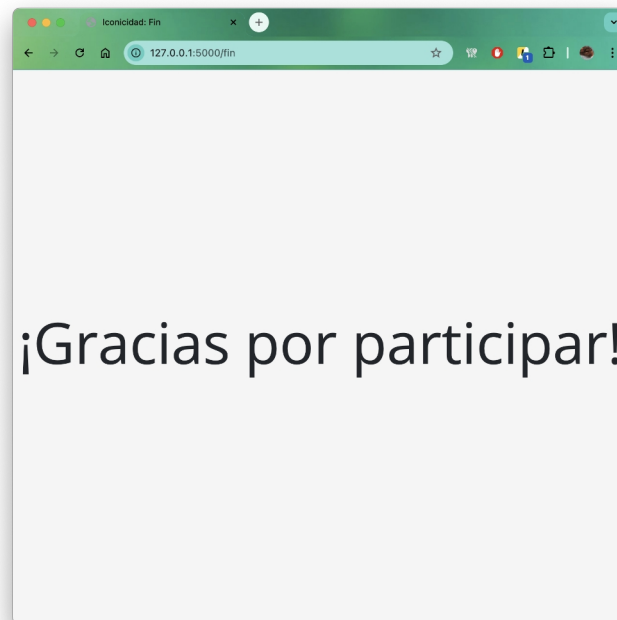
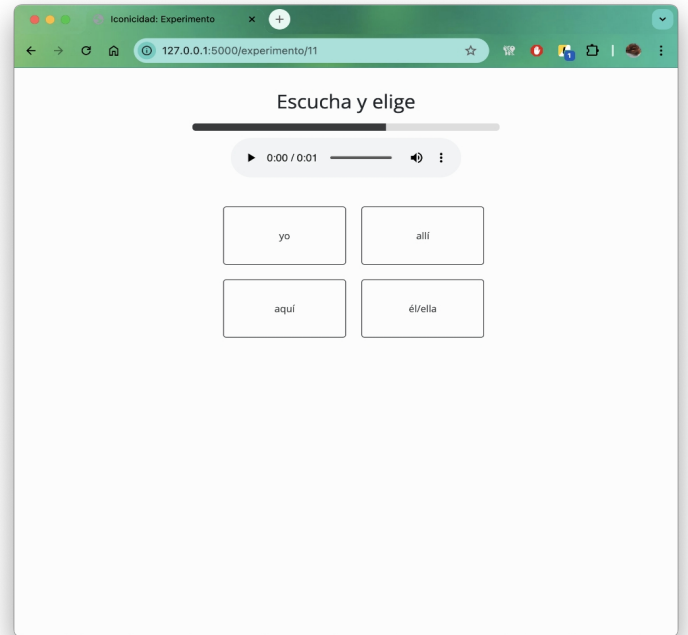
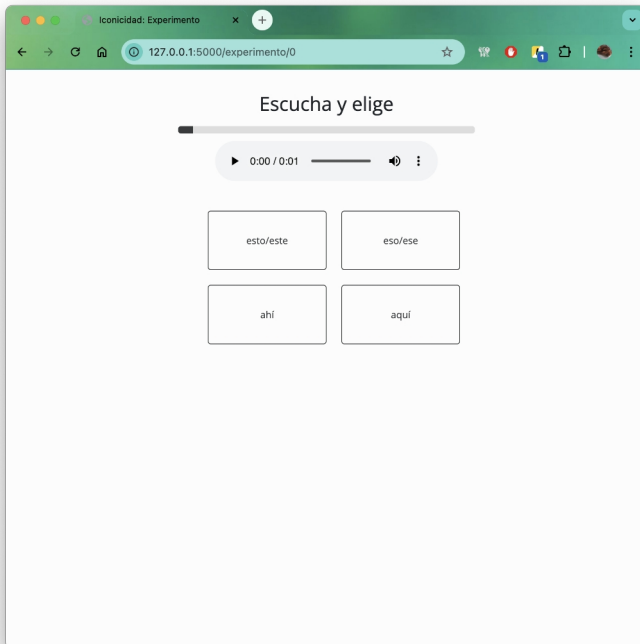
J	MR
30	x Postgrado
x Spanish	
English	

Guardar y continuar

La presentación de instrucciones debe mostrar dos páginas de instrucciones antes del experimento y permitir la navegación secuencial entre estas.



Durante la ejecución del experimento, el sistema debe permitir la randomización del orden de presentación de ítems para cada participante, presentar estímulos auditivos que puedan reproducirse mínimo dos veces, mostrar las cuatro (4) opciones de respuesta textuales para cada ítem y randomizar su orden, mostrar la barra de progreso durante el experimento, y registrar cada respuesta inmediatamente en la base de datos.





Para almacenar los datos el sistema debe direccionar los datos de participantes en la tabla correspondiente, así como almacenar las respuestas con marca de acierto (correcto/incorrecto), además de mantener la integridad referencial entre participantes y respuestas, y de utilizar transacciones para garantizar consistencia de datos.

En el manejo de sesiones el sistema debe mantener el ID del participante y el orden randomizado de ítems durante toda la sesión, y debe prevenir el acceso al experimento sin registro previo.

5.2 Requisitos no funcionales

Los requisitos no funcionales de la página web son la usabilidad, el rendimiento, la portabilidad, la mantenibilidad, la seguridad, y la seguridad.

La usabilidad asegura una interfaz limpia y minimalista con navegación intuitiva sin necesidad de entrenamiento previo, un diseño *responsive*, y el uso de convenciones estándar (botones y formularios).

El rendimiento asegura que el tiempo de carga de cada página sea inferior a un (1) segundo en *localhost*, la reproducción del audio sin *lag* perceptible y el registro de respuestas instantáneo con un tiempo inferior a doscientos (200) milisegundos.

La portabilidad debe garantizar que sea ejecutable en cualquier sistema operativo (Windows, macOS y Linux), que la base de datos se almacene en un archivo .db único, que las dependencias sean gestionadas mediante *requirement.txt*, y que la carpeta del proyecto sea autocontenida y transportable.

La mantenibilidad se cerciora de que el código esté organizado en módulos (*main.py*, *models.py*, *db.py*), que haya comentarios explicativos donde el código sea complejo, que se utilice ORM para abstraer las consultas SQL, y que haya una separación clara entre la lógica de (Python) y la presentación (HTML/Jinja2).

En el caso de la seguridad se usa *secret_key* para las sesiones Flask de cada participante, se valida la entrada de los formularios, el manejo de excepciones con *rollback* en caso de error, y que, al tener un uso local controlado, no se implementa autenticación ni HTTPS.



5.3 Requisitos del entorno

En términos del entorno de trabajo, hay que asegurar que en el software se utilice Python 3.12 o superior, un navegador web moderno (Chrome, Firefox, Safari, Edge), y que el sistema operativo sea Windows 10 o superior, macOS 10.15 o superior, o Ubuntu 20.04 o superior. En lo relativo al hardware el procesador puede ser el de cualquier CPU moderna, con una RAM de mínimo 4 GB, necesidad de almacenamiento de aproximadamente 100 MB para la aplicación y base de datos, y una salida de audio funcional, ya sea a través de altavoces o auriculares.

6. MANUAL DE INSTALACIÓN

6.1 Requisitos previos

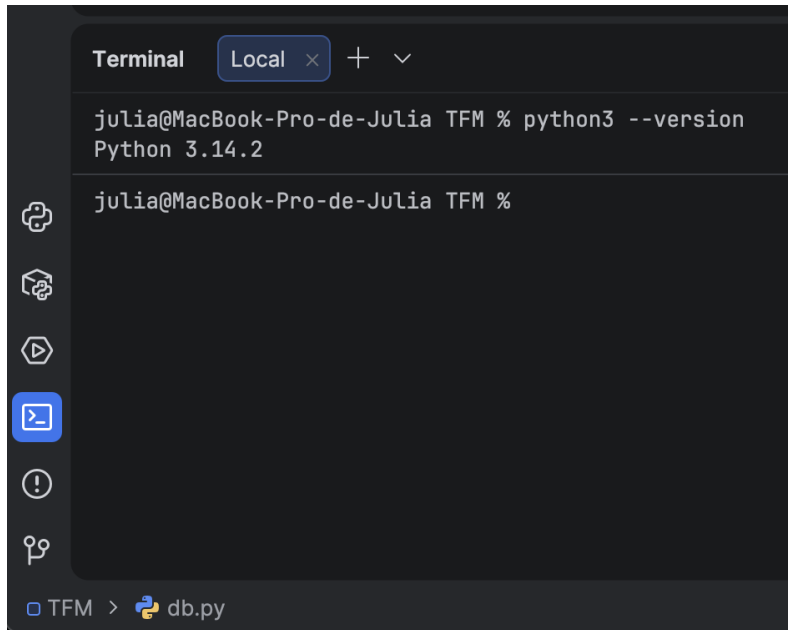
Antes de instalar la aplicación, hay que asegurarse de tener instalado Python 3.12 o superiores, que puede descargarse desde <https://www.python.org/downloads/>. Durante la instalación, marca *Add Python to PATH*. Reinicia si es necesario y desde el terminal de la computadora verifica la instalación:

- Windows: win + R, escribe “cmd” y presiona Enter,
- macOS: cmd + espacio, escribe “Terminal” y presiona Enter,
- Linux: Ctrl + Alt + T.

En el terminal, ejecuta “python –version” (“python3⁴ –version” en el caso de macOS) y debería salir un mensaje similar a “Python 3.12.0” (o superior).

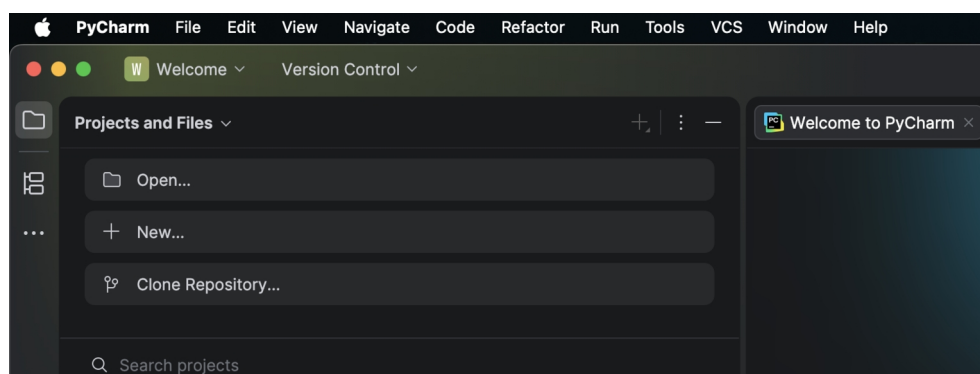
También puedes hacer esta instalación directamente desde el IDE, PyCharm en nuestro caso. Para descargar el IDE, puede hacerse desde la web <https://jetbrains.com/pycharm/download/>. Se selecciona *Community Edition* (que es la versión gratuita) y se instala con las opciones por defecto.

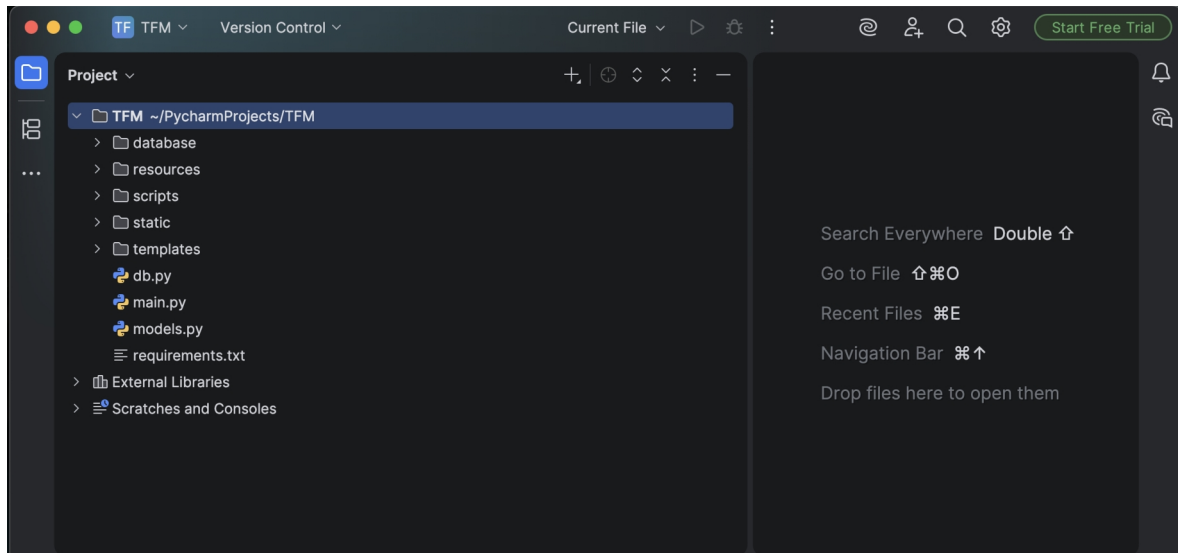
⁴ En macOS, el comando es “python3” y no “python”, por lo que todos los comandos en este manual que usen “python” en este manual han de cambiarse por “python3” y “pip” por “pip3”.



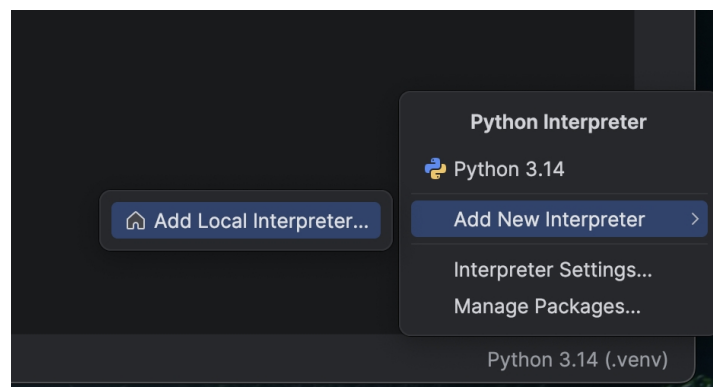
6.2 Instalación paso a paso

1. Abrir el proyecto en PyCharm. En la carpeta de bienvenida, hacer clic en *Open* y navega hasta llegar a la carpeta donde tengas el proyecto guardado. Lo seleccionas y haces click en *OK* para que PyCharm abra el proyecto, donde mostrará todos los archivos en el panel izquierdo.

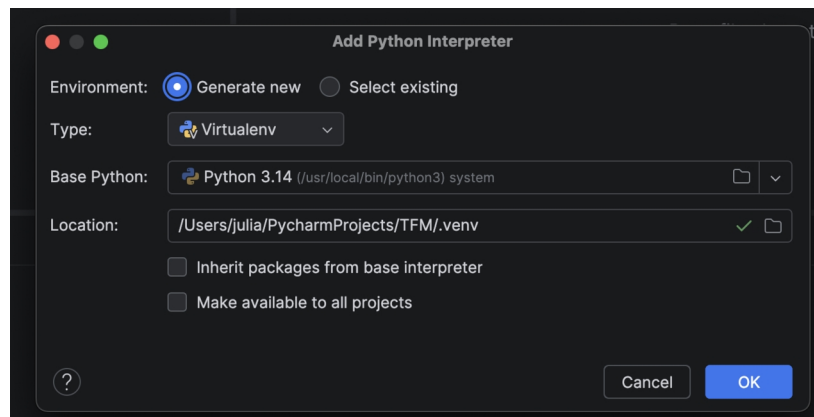




2. Para configurar el intérprete de Python, busca en la esquina inferior derecha el texto *No Interpreter* o *Python.X.X*. Cliquea ahí y selecciona *Add New Interpreter* → *Add Local Interpreter*.

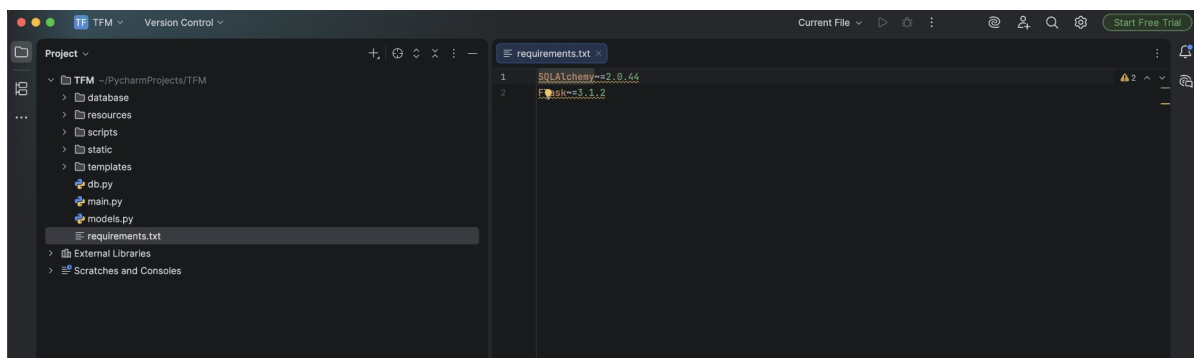


En la ventana que aparece, clic en *Virtual Environment*⁵ en el panel izquierdo, marca *New Environment* y, en *Base Interpreter*, selecciona la versión de python que instalaste. Deja la ubicación por defecto y haz click en OK. Puede que tarde 1-2 minutos en crear el entorno virtual.



3. Instalamos las dependencias Flask y SQLAlchemy:

En el panel izquierdo de PyCharm buscamos el archivo “requirements.txt”. Hacemos click derecho sobre él y seleccionamos *Install All Packages*, con lo que se realizará una instalación automática de Flask y SQLAlchemy. Después hay que esperar a que aparezca el mensaje *Package installed successfully* en la parte inferior.



De forma alternativa, si no aparece la opción de *Install All Packages*, haz click en el terminal en el menú inferior y ejecuta los siguientes comandos:

- Windows y Linux: `pip install -r requirements.txt`
- macOS: `pip3 install -r requirements.txt`

⁵ El entorno virtual es una carpeta aislada donde se instalan las dependencias del proyecto (Flask, SQLAlchemy) sin afectar al lenguaje de programación del sistema (Python). Esto previene conflictos entre diferentes proyectos.

```
Terminal Local + -
julia@MacBook-Pro-de-Julia TFM % pip3 install -r requirements.txt
Collecting SQLAlchemy==2.0.44 (from -r requirements.txt (line 1))
  Downloading sqlalchemy-2.0.44-py3-none-any.whl.metadata (9.5 kB)
Collecting Flask==3.1.2 (from -r requirements.txt (line 2))
  Downloading flask-3.1.2-py3-none-any.whl.metadata (3.2 kB)
Requirement already satisfied: greenlet<=1 in /Library/Frameworks/Python.framework/Versions/3.14/lib/python3.14/site-packages (from SQLAlchemy==2.0.44->-r requirements.txt (line 1)) (3.3.0)
Requirement already satisfied: typing-extensions>=4.6.0 in /Library/Frameworks/Python.framework/Versions/3.14/lib/python3.14/site-packages (from SQLAlchemy==2.0.44->-r requirements.txt (line 1)) (4.15.0)
Collecting blinker==1.9.0 (from Flask==3.1.2->-r requirements.txt (line 2))
  Using cached blinker-1.9.0-py3-none-any.whl.metadata (1.6 kB)
Collecting click==8.1.3 (from Flask==3.1.2->-r requirements.txt (line 2))
  Using cached click-8.1.3-py3-none-any.whl.metadata (2.6 kB)
Collecting itsdangerous==2.2.0 (from Flask==3.1.2->-r requirements.txt (line 2))
  Using cached itsdangerous-2.2.0-py3-none-any.whl.metadata (1.9 kB)
Collecting jinja2==3.1.2 (from Flask==3.1.2->-r requirements.txt (line 2))
  Using cached jinja2-3.1.2-py3-none-any.whl.metadata (2.9 kB)
Collecting markupsafe==2.1.1 (from Flask==3.1.2->-r requirements.txt (line 2))
  Using cached markupsafe-2.1.1-cp314-cp314-macosx_10_13_x86_64.whl.metadata (2.7 kB)
Collecting werkzeug==3.1.0 (from Flask==3.1.2->-r requirements.txt (line 2))
  Downloading werkzeug-3.1.0-py3-none-any.whl.metadata (4.0 kB)
  Downloading sqlalchemy-2.0.44-py3-none-any.whl (1.9 MB)
    1.9/1.9 MB 3.0 MB/s 0:00:00
  Downloading flask-3.1.2-py3-none-any.whl (103 kB)
  Using cached blinker-1.9.0-py3-none-any.whl (8.5 kB)
  Using cached click-8.1.3-py3-none-any.whl (108 kB)
  Using cached itsdangerous-2.2.0-py3-none-any.whl (16 kB)
  Using cached jinja2-3.1.2-py3-none-any.whl (134 kB)
  Using cached markupsafe-2.1.1-cp314-cp314-macosx_10_13_x86_64.whl (11 kB)
  Downloading werkzeug-3.1.0-py3-none-any.whl (225 kB)
Installing collected packages: SQLAlchemy, markupsafe, itsdangerous, click, blinker, werkzeug, jinja2, Flask
Successfully installed Flask-3.1.2 SQLAlchemy-2.0.44 blinker-1.9.0 click-8.1.3 itsdangerous-2.2.0 jinja2-3.1.2 markupsafe-2.1.1 werkzeug-3.1.0

[notice] A new release of pip is available: 25.3 -> 26.0.1
[notice] To update, run: pip3 install --upgrade pip

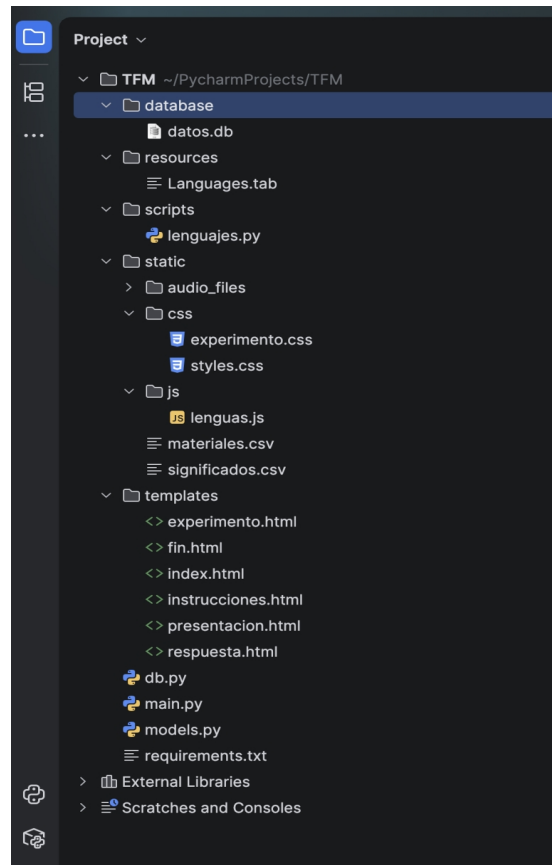
julia@MacBook-Pro-de-Julia TFM %
```

Y espera a que termine la instalación.

4. Verificar la estructura del proyecto.

En el panel izquierdo de

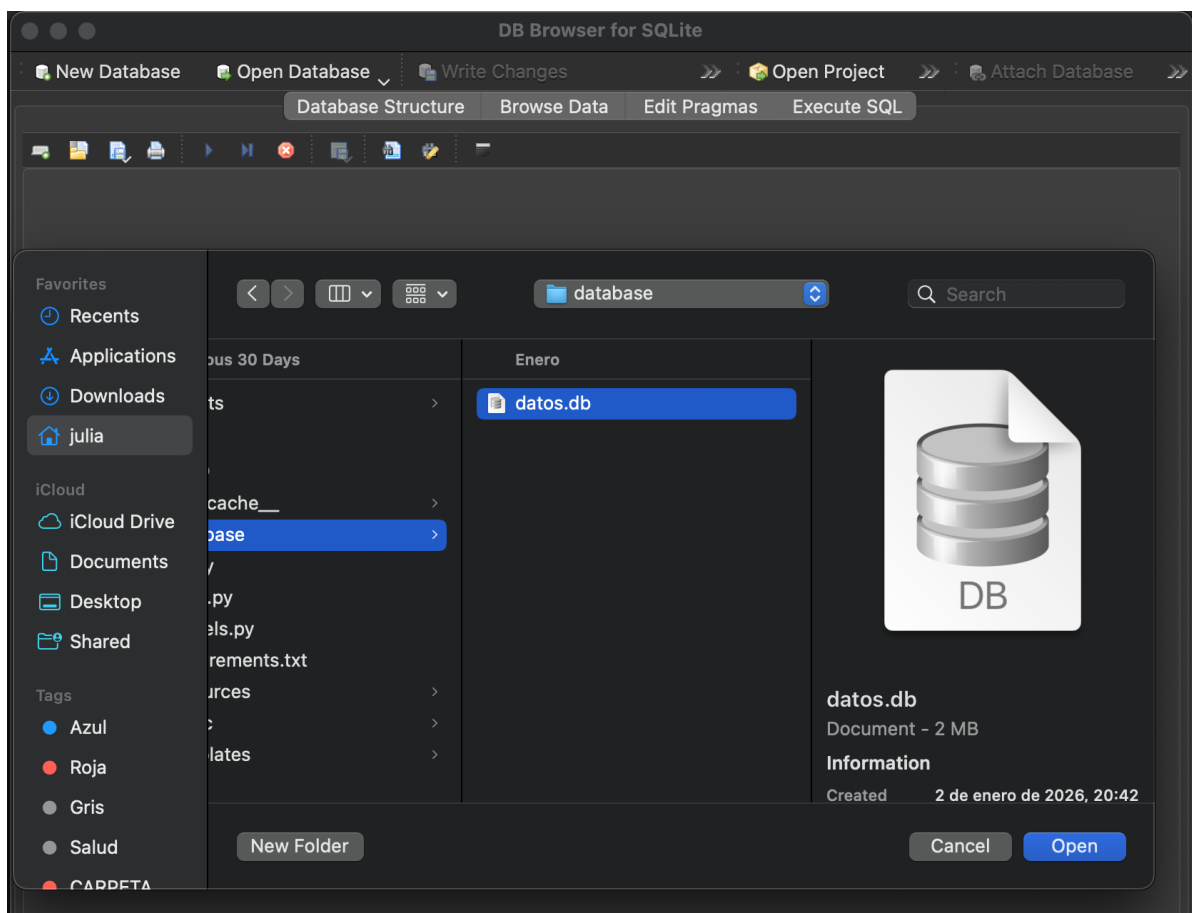
PyCharm deberías ver lo siguiente:



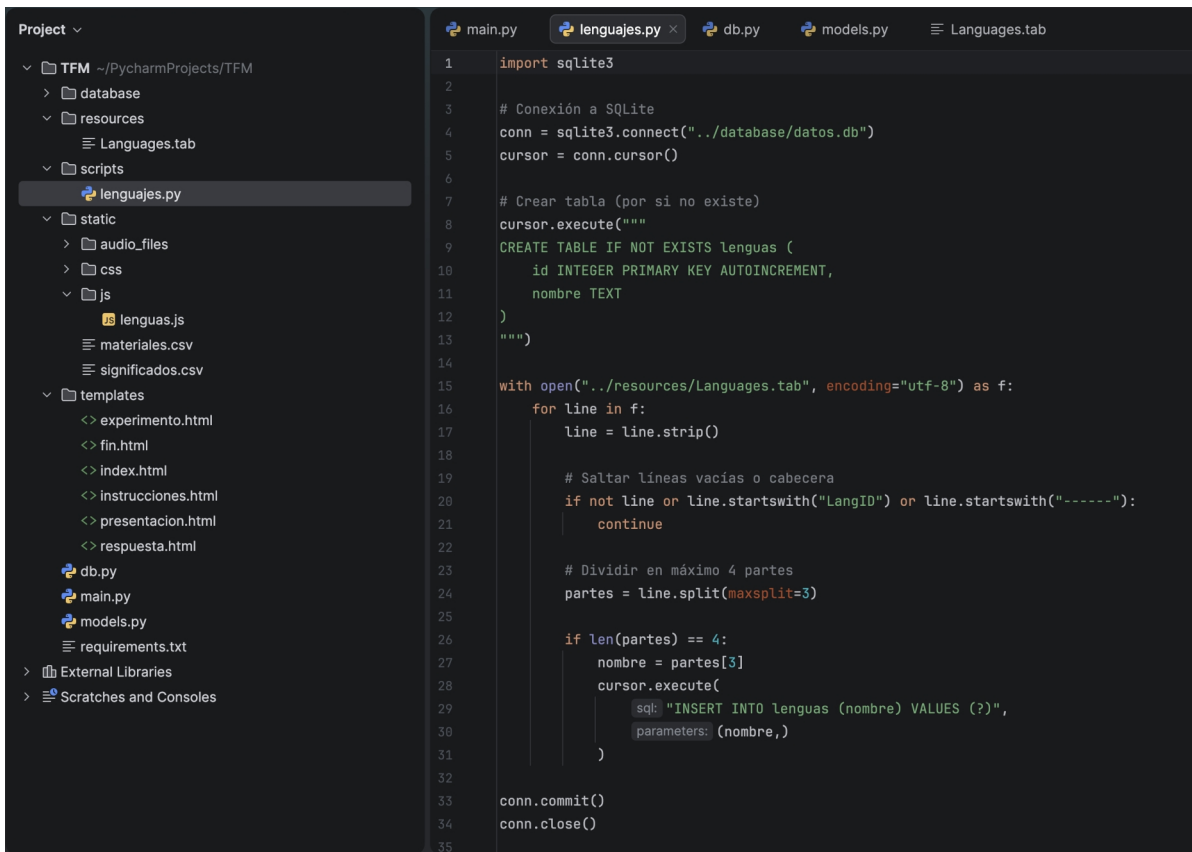
6.3 Configuración de la Base de Datos (BBDD)

La base de datos SQLite se crea automáticamente la primera vez que se ejecuta la aplicación, por lo que no requiere configuración manual.

Si la tabla “lenguas” está vacía, hay que importarla desde DB Browser. Descarga el programa desde <https://sqlitebrowser.org/> e instala el proyecto. Al abrir el programa, haz click en *Open Database* y navega hasta la base de datos del proyecto (datos.db).

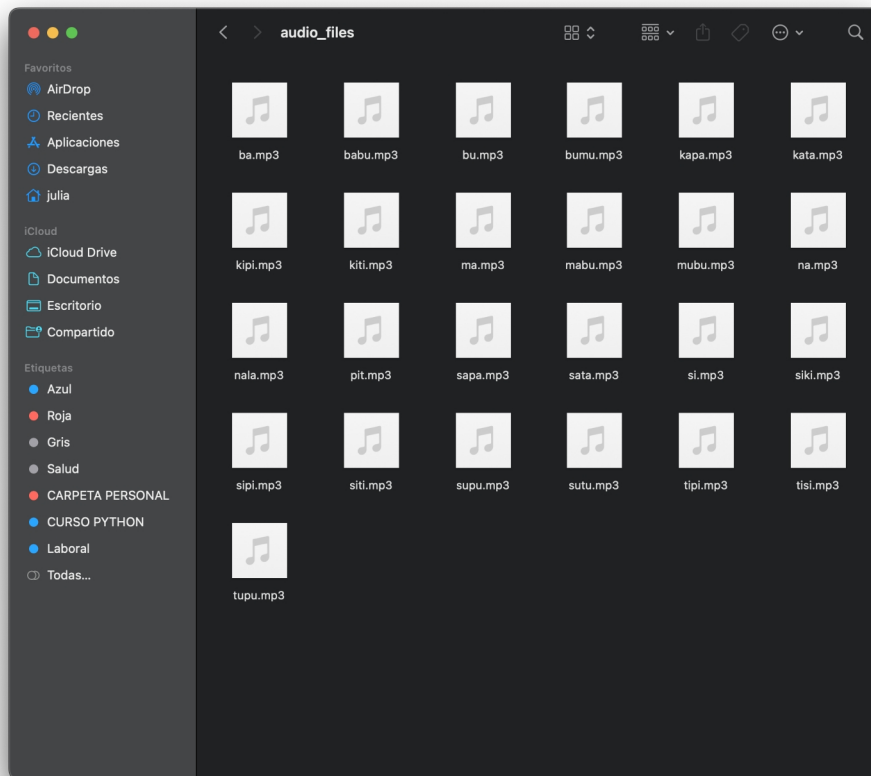


En el caso de que no funcione la importación, realiza el código indicado en la siguiente captura, en la carpeta denominada “/scripts”, donde se almacenarán los datos exportados de ETHNOLOGUE en “Languages.tab” en un fichero “.py” que se iniciará automáticamente cuando el programa empiece a correr.



```
1 import sqlite3
2
3 # Conexión a SQLite
4 conn = sqlite3.connect("../database/datos.db")
5 cursor = conn.cursor()
6
7 # Crear tabla (por si no existe)
8 cursor.execute("""
9 CREATE TABLE IF NOT EXISTS lenguas (
10     id INTEGER PRIMARY KEY AUTOINCREMENT,
11     nombre TEXT
12 )
13 """)
14
15 with open("../resources/Languages.tab", encoding="utf-8") as f:
16     for line in f:
17         line = line.strip()
18
19         # Saltar líneas vacías o cabecera
20         if not line or line.startswith("LangID") or line.startswith("-----"):
21             continue
22
23         # Dividir en máximo 4 partes
24         partes = line.split(maxsplit=3)
25
26         if len(partes) == 4:
27             nombre = partes[3]
28             cursor.execute(
29                 sql: "INSERT INTO lenguas (nombre) VALUES (?)",
30                 parameters: (nombre,)
31             )
32
33 conn.commit()
34 conn.close()
35
```

Para verificar que todos los archivos .mp3 referenciados en static/materiales.csv existen, abre el archivo static/audio_files/. Los nombres deben coincidir a la perfección, incluyendo mayúsculas/minúsculas.



6.4 Ejecución del experimento

A través del IDE (PyCharm), podemos ejecutar el experimento de dos formas distintas:

A) Opción 1: ejecutar desde el Terminal integrado:

En PyCharm, haz click en *Terminal* en la parte inferior y ejecutar:

- Windows y Linux: `python main.py`
- macOS: `python3 main.py`



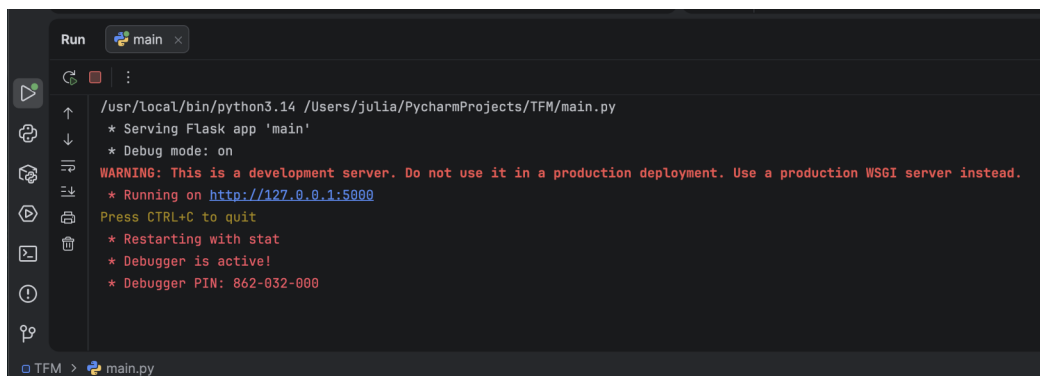
```
Terminal Local x + v
julia@MacBook-Pro-de-Julia TFM % python3 main.py
* Serving Flask app 'main'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 862-032-000

TFM > main.py
```

B) Opción 2: ejecutar desde PyCharm (recomendado)

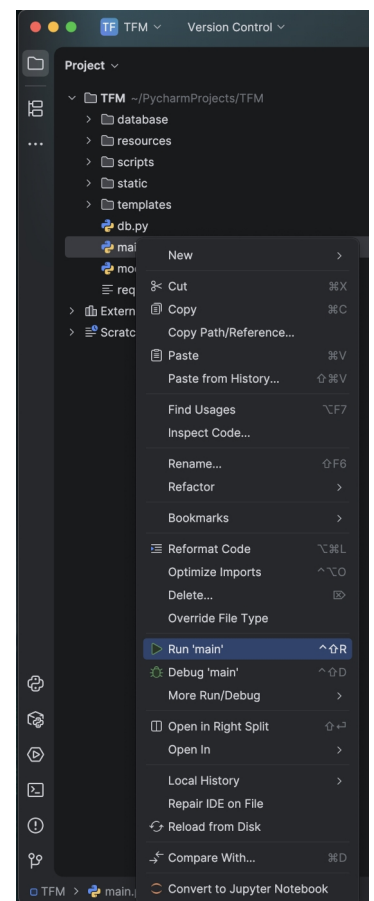
En el panel izquierdo, hacer click derecho sobre el archivo “main.py” y seleccionar “Run ‘main’”. Aparecerá la consola en el panel inferior.

Hay que mantener esta ventana abierta para que el servidor siga ejecutándose en cada sesión en la que se registren datos experimentales.



```
Run main x
/usr/local/bin/python3.14 /Users/julia/PycharmProjects/TFM/main.py
* Serving Flask app 'main'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 862-032-000

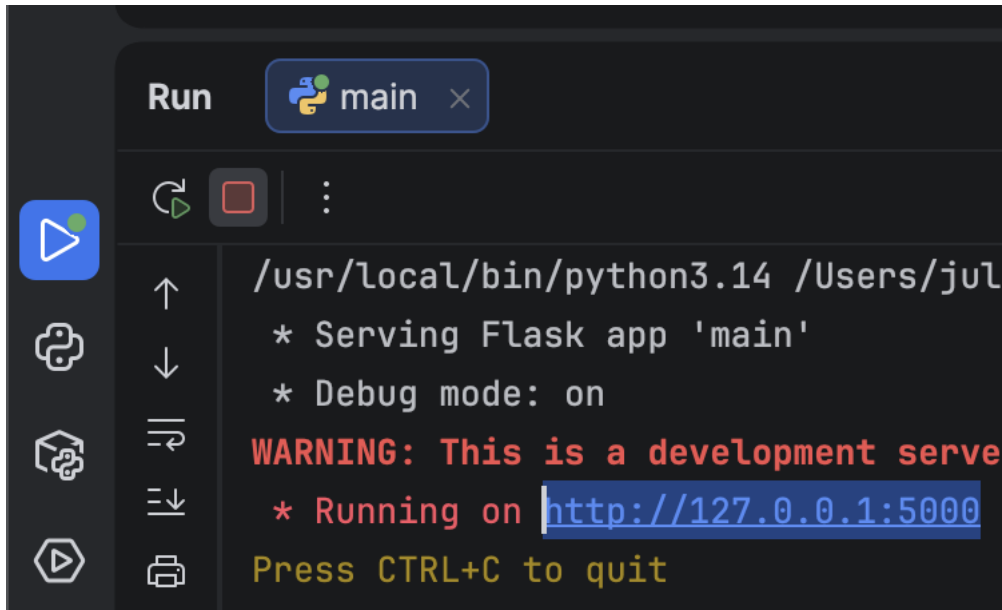
TFM > main.py
```



Para acceder al experimento, podemos clicar directamente en el enlace que aparece en la consola, o abre cualquier navegador web y pegar en la barra de direcciones la dirección indicada en el terminal (<http://127.0.0.1:5000>). Después, realiza el experimento tal cual indican las instrucciones.



Al finalizar la sesión de registro de datos experimentales, volvemos a la ventana de PyCharm y hacemos clic en el botón rojo de *Stop* (cuadrado rojo) en la esquina superior izquierda del panel de la consola, o presiona Ctrl + C, como indica la propia consola.



```
Run  main x
[Run button] [Stop button] [Menu]
↑ /usr/local/bin/python3.14 /Users/jul
↓ * Serving Flask app 'main'
⇐ * Debug mode: on
⇐ WARNING: This is a development server
⇐ * Running on http://127.0.0.1:5000
⇐ Press CTRL+C to quit
[Print button]
```

6.5 Solución de problemas comunes

6.5.1 Error: No interpreter configured

Este error nos indica que PyCharm no encuentra Python. La solución es repetir el punto 2 del apartado 6.2 para configurar el intérprete.

6.5.2 Error: Module not found (Flask, SQLAlchemy)

Es decir, las dependencias no están instaladas. Hay que repetir el punto 3 del apartado 6.2 para instalarlas desde “requirements.txt”.

6.5.3 Error: Puerto 500 ya en uso

Otra aplicación está utilizando ya el puerto 500, por lo que hay que detener cualquier otra instancia de Flask que esté corriendo o cambiar el puerto en main.py. Para ello, en el archivo “main.py” añade al final del todo “app.run(debug=True, port=5001)”.



6.5.4 Error: No se reproduce el audio

Este error nos indica que los archivos de audio no se encuentran o que el navegador no tiene soporte para ellos. Para solucionarlo hay que verificar que los archivos .mp3 existen en la carpeta static/audio_files/, que el volumen del sistema no esté silenciado, o que los nombres en materiales.csv coinciden con exactitud con los archivos .mp3. También es recomendable utilizar un navegador moderno (Chrome, Firefox, Safari, Edge).

6.5.5 Error: Base de datos bloqueada

Debido a que DB Browser tiene abierto el archivo “datos.db” no se puede actualizar la base de datos. Simplemente termina las tareas que estuvieras realizando en DB Browser y cierra el programa antes de ejecutar la aplicación de nuevo.

6.5.6 PyCharm no encuentra la carpeta del proyecto

El disco externo o el USB están desconectados, por lo que es recomendable volver a conectarlo o, en su lugar, copiar la carpeta al disco local (aunque no actualizaría las mismas BBDD).

7. CONCLUSIONES

En esta sección comentamos los logros, dificultades y aprendizajes durante la realización del proyecto.

Podemos decir que, de forma exitosa, se implementaron todas las funcionalidades planificadas. El experimento funcionó desde las pruebas piloto, por lo que la conexión entre la tabla de materiales y los audios ha sido correcta en todo momento. Este proyecto ha permitido la recolección de 20 respuestas de distintos participantes (1 nula, el participante abandonó la sesión antes de completar el cuestionario), aportando unos primeros datos útiles para el análisis de la teoría.

En términos de dificultades, el primer obstáculo fue la obtención de la lista de lenguas, sacadas de la web lingüística ETHNOLOGUE y exportadas en formato .tab. Después, hubo que revisar el código porque esta lista se duplicaba con cada sesión/participante que completaba el cuestionario. Así mismo, la lista de idiomas aparece en inglés (idioma de la fuente), mientras que el experimento se realiza en español. Dado el alto nivel de alfabetización en inglés en la población española, se



decidió que no era primordial su traducción al español y, en caso de duda, al tratarse de un experimento de laboratorio, el participante siempre podía solicitar ayuda por parte del profesional presente.

Seguidamente, los problemas que tomaron más tiempo en solucionarse surgieron en la implementación de HTML y CSS debido a la falta de experiencia con ambos lenguajes. Tanto es así, que ha habido aspectos que no han podido solucionarse por falta de conocimiento (el texto del *dropdown* de “otras lenguas” se mantiene a la derecha, en vez de estar a la izquierda), pero que se han solventado con imaginación (se cambió el texto por uno más breve e igual de intuitivo). Otras dificultades encontradas han sido la funcionalidad de la barra de progreso, centrar los textos de las opciones del experimento, y asegurar la randomización, tanto de los ítems como de la posición de las respuestas.

A lo largo de todo el código hay comentarios complementarios o explicativos de por qué se utiliza un método u otro, lo que asegura una buena práctica documental y que, en caso de reutilizar el código, se sepa qué puede modificarse y qué no.

8. EVOLUTIVOS DEL PROYECTO

En esta sección propongo futuras mejoras para la página web en cuanto a la interfaz de usuario, el diseño experimental, la gestión de datos y las técnicas usadas.

Algunas mejoras en la interfaz de usuario sería la implementación del tema oscuro para reducir la fatiga visual en experimentos largos, mejorar la accesibilidad mediante la compatibilidad con lectores de pantalla, atajos de teclado para la navegación, y un mayor contraste de colores (cumplir con WCAG 2.1), crear una interfaz multilingüe que permita a los participantes elegir el idioma de la interfaz (dependiendo del público objetivo del experimento o del lugar de desarrollo del experimento) sin cambiar el idioma de los estímulos experimentales.

En términos de mejoras del diseño experimental, se pueden utilizar estímulos visuales en lugar de texto mediante imágenes que representen los conceptos deícticos (cerca/lejos), vídeos cortos que muestren dichos gestos deícticos o diagramas espaciales interactivos, lo que eliminaría posibles sesgos de lectoescritura y permitiría evaluar la iconicidad de forma más pura, sin mediación del texto escrito. Otro dato que podría almacenarse es el registro de tiempos de reacción



desde la presentación del estímulo hasta la respuesta del participante, lo que podría revelar diferencias en la velocidad de procesamiento entre asociaciones icónicas y no icónicas. De modo alternativo, una escala de Likert (1-5 o 1-7) donde el participante indique cuán seguro está de su respuesta permitiría distinguir entre respuestas intuitivas y dudosas.

Para la gestión de datos se podría crear un panel de administración en una interfaz web para visualizar los participantes y sus respuestas, exportar los datos a CSV o Excel para realizar un análisis estadístico, y una visualización de estadísticas básicas que vemos a través de la vista de DB Browser, que el sistema genere copias de seguridad de la base de datos de forma automática cada N participantes o cada X número de usos, y la anonimización de nombres completos antes de compartir datos para el análisis, cumpliendo así con la RGPD si se expande fuera del laboratorio.

En cuanto a las mejoras técnicas, si el proyecto escala a múltiples laboratorios o experimentadores se debería migrar a PostgreSQL para facilitar el acceso concurrente de múltiples usuarios, mejorar el rendimiento con grandes volúmenes de datos y con características avanzadas (*triggers, views, full-text search*). También se puede empaquetar la aplicación en un contenedor Docker para facilitar su despliegue en diferentes entornos sin problemas de dependencias, y el uso de API REST para separar *frontend* de *backend*, lo que permitiría el desarrollo de aplicaciones móviles nativas (iOS/Android), la integración con otras herramientas de análisis, y la reutilización del *backend* para otros experimentos.